RAJALAKSHMI ENGINEERING COLLEGE

Department of Computer Science and Engineering (Cyber Security)

CS23533– Foundations of Artificial Intelligence

2026-2027

AI-Based Cyber Defence Allocation Using Game Theory

Supervisor
Dr. Bhuvana S

Presented by
Puvanesu R
241901085

Problem Statement and Motivation

- The game addresses network defence, where a limited security budget must be spread across many vulnerable digital assets.
- Attackers pick off whatever stays exposed, so every protect, scan or pass decision reshapes the whole risk landscape.
- The project shows how adversarial search, Minimax and Alpha-Beta pruning, models a real attacker-defender struggle.
- A live worst-case damage forecast, search statistics and a reasoning trail make every AI decision easy to inspect.

Objectives

- Model the attack-defence struggle as a two-player, zero-sum, turn-taking game on a board of assets.
- Define value and vulnerability per asset; risk = value x vulnerability drives every decision.
- Implement Minimax with Alpha-Beta pruning, move ordering and a transposition table over bitmask states.
- Give the Defender budget-limited protect, scan and pass actions against an Attacker who maximises damage.
- Compare nodes expanded, cutoffs and cache hits, and forecast worst-case damage live during play.

Abstract

- The Cyber Security Game is a two-player security strategy battle built entirely with classical AI. A Defender protects high-risk digital assets on a limited budget while an Attacker breaches whatever is left exposed, and the higher score wins the match.
- Every scenario samples real assets from a 22-asset pool; each asset carries a value and a vulnerability, and their product gives the risk both sides fight over. Expert moves come from Minimax with Alpha-Beta pruning over bitmask game states, using high-risk-first move ordering and a transposition table with exact and bound values.
- The Tkinter console shows a live worst-case damage forecast with node counts, an AI Brain Battle chart comparing both algorithms, and a move-by-move reasoning trail. Human vs AI, local two-player and AI vs AI modes are supported. No machine learning is used.

Literature Survey

No.	Author(s) & Year	Paper Title	Methodology / Algorithm	Limitations
1	Russell & Norvig (2021)	Artificial Intelligence: A Modern Approach (4th ed.)	Minimax with Alpha-Beta pruning, evaluation functions, game theory	Textbook two-player forms. The game adapts them to budgeted security defence.
2	Shannon (1950)	Programming a Computer for Playing Chess	Minimax search with an evaluation function over game trees	Fixed-depth chess evaluation. Budgets and asset risk are not modeled.
3	Lye & Wing (2005)	Game Strategies in Network Security	Two-player game model of attacker versus network administrator	Continuous stochastic models; hard to solve exactly and not playable.

Literature Survey

No.	Author(s) & Year	Paper Title	Methodology / Algorithm	Limitations
4	Knuth & Moore (1975)	An Analysis of Alpha-Beta Pruning	Alpha-Beta pruning effectiveness with move ordering	Savings depend on ordering. The game tries high-risk assets first.
5	Slate & Atkin (1977)	CHESS 4.5 – The Northwestern University Chess Program	Transposition table caching subtree values for reuse	Memory grows with the tree; bound flags needed for cutoff values.
6	Manshaei et al. (2013)	Game Theory Meets Network Security and Privacy	Survey of attacker-defender game models (ACM CSUR)	Theoretical survey; few open, playable or explainable implementations.

Research Gap/Existing System

1. What existing studies do
Minimax and Alpha-Beta solve two-player games; security games model attacker-defender conflict.

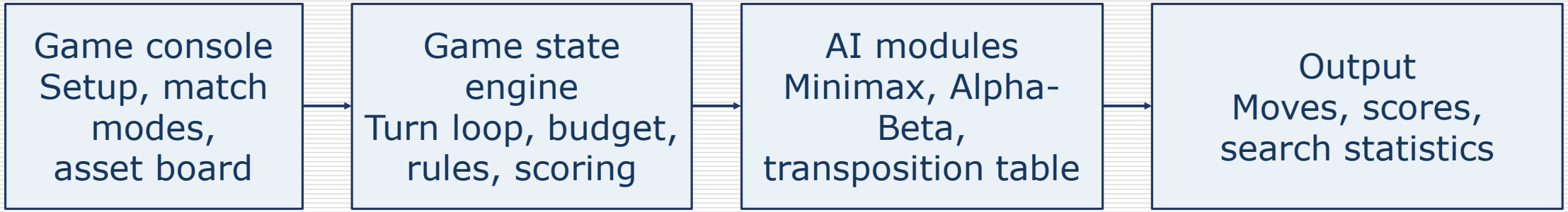
2. Limitations observed
Chess-style games ignore budgets; security models are rarely playable or explainable.

3. Research gap
One open prototype should turn adversarial search into a measurable security battle.

4. How our project responds
The game pairs both algorithms with budgeted defence and a live reasoning trail.

Proposed System

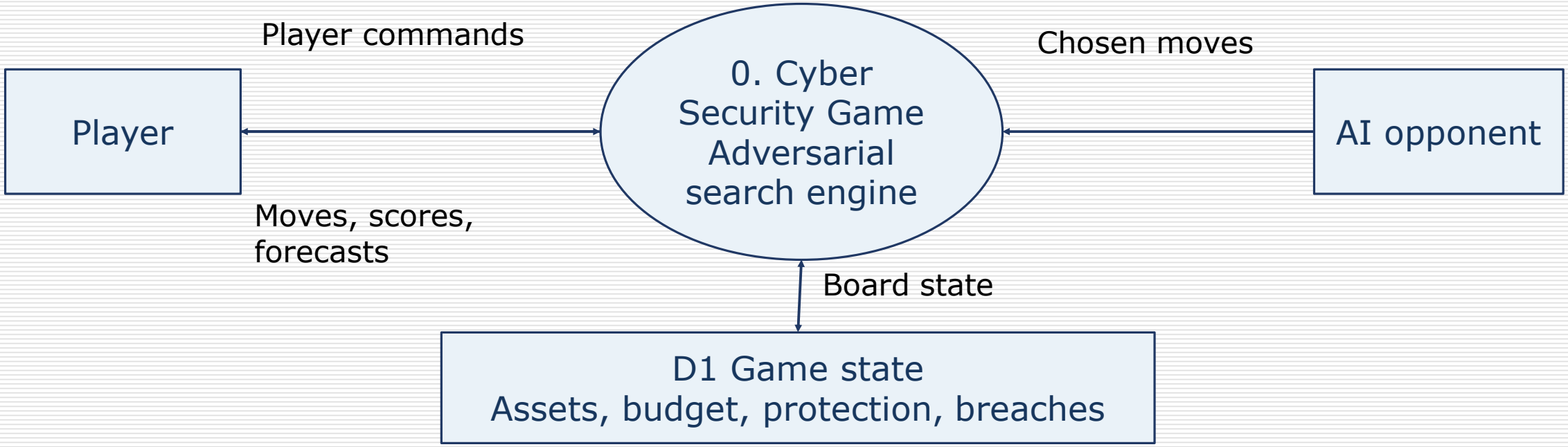
Proposed architecture



Rules live in a pure GameState class; search runs on cloned bitmask states, so hypothetical moves never corrupt the board.
The asset pool holds 22 realistic digital assets; every seed reshuffles a fresh 6 to 10 asset scenario.

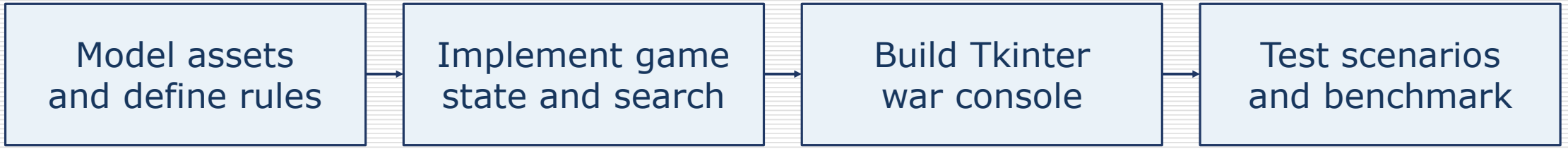
DFD Diagram

Level 0 context diagram

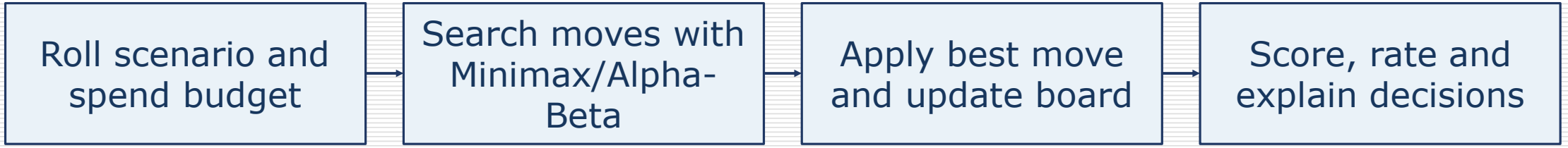


Methodology/Work Flow Diagram

Development workflow



Runtime workflow



The worst-case damage readout re-runs Alpha-Beta after every move, showing the exact minimax forecast.

Tools and Technologies

- Python 3 implements the game rules, Minimax and Alpha-Beta search from scratch, with no AI libraries.
- Tkinter and ttk build the console: asset board, command menu, intel popups and game-over screen.
- Random and dataclasses shape seeded scenarios; difflib matches mistyped asset names.
- The engine counts nodes, cutoffs and cache hits; VS Code supports development.
- Standard laptop: 4 GB RAM minimum, no GPU. No database or machine learning is used.

List of Modules

- Module 1 – Asset and Scenario Model: 22 assets with value, vulnerability and risk.
- Module 2 – Game State and Rules: turn loop, budget, actions and scoring.
- Module 3 – Minimax Search: plain adversarial recursion with caching.
- Module 4 – Alpha-Beta Search: pruning, move ordering, transposition table.
- Module 5 – Search Instrumentation: nodes, cutoffs, cache hits, comparison.
- Module 6 – AI Turn Controller: Novice random moves vs Expert search.
- Module 7 – Game Console and Popups: board, command menu, intel, banter.
- Module 8 – Game Over and Analysis: rating, node chart, reasoning, replay.

Module Descriptions – 1 to 3

- Module 1 – Asset and Scenario Model
- Input: asset pool and seed. Risk = value x vulnerability over 22 assets; 6 to 10 sampled per match.
- Module 2 – Game State and Rules
- Input: player actions. Protect, scan, pass, attack and end turn; budget and stalemate rules. Output: scores.
- Module 3 – Minimax Search
- Input: cloned state masks. Defender minimises terminal damage, Attacker maximises it. Output: best move.
- Status: modules 1 to 3 implemented and tested.

Module Descriptions – 4 to 6

- Module 4 – Alpha-Beta Search: window pruning with high-risk-first move ordering and EXACT, LOWER, UPPER bound caching in a transposition table.
- Module 5 – Search Instrumentation: counts nodes, cutoffs and cache hits; AI Brain Battle compares Minimax and Alpha-Beta on the same start state.
- Module 6 – AI Turn Controller: Expert runs the chosen algorithm for either role; Novice picks a random legal move.
- Status: modules 4 to 6 implemented and tested.

Module Descriptions – 7 to 8

- Module 7 – Game Console and Popups: asset table with live status, command menu with typed commands, asset intel popups, banter and pause menu.
- Module 8 – Game Over and Analysis: winner by secured risk vs damage, S to D rating, node chart, reasoning trail and replay.
- Status: modules 7 and 8 implemented and tested. Console screenshots are to be added.

Results & Discussion

- Same start state, 8 assets, budget 4: Minimax 85,944 nodes; Alpha-Beta 3,644 (95.8% fewer) with 864 cutoffs.
- Both chose identical moves with the same worst-case damage (223.6).
- Scaling 6 to 10 assets lifted savings from 89.5% to 98.3% (0.26 s vs 5.7 s at 10 assets).
- Full AI vs AI match, seed 424242: Defender won 242.6 to 223.6 in 4 rounds on 5,495 nodes.
- Baseline: plain Minimax. The live worst-case forecast stays instant via Alpha-Beta after every move.

User Interface/ Screenshots

- Menu: match mode (vs AI, local two-player, AI vs AI), role, difficulty, algorithm, seed and asset count.
- Console: asset board with exposed, scanned, protected or breached status, budget, scores, worst-case damage.
- Command menu: protect, scan, attack, pass and typed commands; asset intel popups show value, risk and status.
- Working application screenshots: [insert verified screenshots of the console and game-over screen].

Conclusion

- The Cyber Security Game applies classical adversarial search to security decision-making without machine learning.
- Minimax and Alpha-Beta with move ordering and a transposition table drive Expert play for both sides of a zero-sum battle.
- Alpha-Beta matched plain Minimax move-for-move on up to 98% fewer nodes, keeping the live forecast instant.
- The budget rule makes protection a real trade-off; search statistics and a reasoning trail explain every move. Bigger boards remain future work.

Future Work

- Benchmark Minimax and Alpha-Beta on 12+ asset boards with iterative deepening.
- Hide true risks until scanned, so information gathering has measurable value.
- Add chance events such as patch failures and zero-days to move from Minimax to expectiminimax.
- Compare Novice and Expert win rates across many seeded matches.
- Capture screenshots, complete documentation and rehearse the final demonstration.

References

[1] Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
[2] Shannon, C. E. (1950). Programming a computer for playing chess. Philosophical Magazine, 41(314), 256–275.
[3] Knuth, D. E., & Moore, R. W. (1975). An analysis of alpha-beta pruning. Artificial Intelligence, 6(4), 293–326.
[4] Slate, D. J., & Atkin, L. R. (1977). CHESS 4.5 – The Northwestern University chess program. In Chess Skill in Man and Machine (pp. 82–118). Springer.
[5] Lye, K., & Wing, J. M. (2005). Game strategies in network security. International Journal of Information Security, 4(1–2), 71–86.
[6] Manshaei, M. H., Zhu, Q., Alpcan, T., Basar, T., & Hubaux, J.-P. (2013). Game theory meets network security and privacy. ACM Computing Surveys, 45(3), 1–39.
[7] Python Software Foundation. Python documentation. https://docs.python.org/
[8] Python Software Foundation. Tkinter documentation. https://docs.python.org/3/library/tkinter.html

Thank You

Cyber Security Game
Defender vs Attacker security strategy battle